Tech Challenge Plano construtivo

Carlos Frederico Ribeiro de Almeida Dominguez

Planejamento construtivo · Portal de entrega · Vercel + Supabase

Versão 1.0 - PROPOSED

Documento de planejamento. Implementação, testes, vídeo e entrega oficial permanecem pendentes.

01 · Escopo e critérios de entrega

02 · Stack e arquitetura proposta

03 · Domínio e propriedade dos dados

04 · Fluxo de negócio e máquina de estados

05 · Regras de negócio verificáveis

06 · Dados, contratos e segurança

07 · Etapas de construção e critérios de saída

08 · Plano de testes e evidências

09 · Documentação DDD e roteiro do vídeo

10 · Estrutura, operação e conclusão

Visão do fluxo de negócio

Exceção proposta: rejeição mantém Aguardando aprovação e bloqueia execução. Reabrir diagnóstico gera nova versão; aprovação reserva estoque e o início da execução ocorre por comando separado.

01 / Escopo e critérios de entrega

Objetivo: construir o MVP do back-end de uma oficina mecânica, com ordens de serviço, clientes, veículos, serviços, peças, estoque e orçamento. Fonte: Fase 1 - Tech Challenge, PDF consultado no portal FIAP em 25/10/2026. Prazo exibido na atividade: 03/11/2026 às 23h59. O desafio vale 90% da nota da fase.

Este pacote é um plano construtivo e portal documental. Não contém o back-end implementado, resultados de testes, análise de vulnerabilidades executada ou vídeo publicado. Todas essas evidências permanecem pendentes.

Exigências FIAP: monólito em camadas, REST, JWT administrativo, validação de CPF/CNPJ e placa, testes unitários e de integração, cobertura mínima de 80% nos domínios críticos, Dockerfile, Compose e execução local documentada.

Entrega oficial: um único PDF com identificação, links completos e clicáveis, branch e hash final. GitHub público e vídeo público de até 15 minutos durante correção e revisão de notas. O portal complementa essa entrega; não substitui o PDF.

Identificação: Carlos Frederico Ribeiro de Almeida Dominguez; curso SOAT - Software Architecture; fase 1; grupo exibido 144. RM e username Discord: a preencher e conferir. Política de IA: não explicitada no enunciado; confirmação institucional pendente.

02 / Stack e arquitetura proposta

PROPOSED: portal estático HTML/CSS/JavaScript na Vercel; back-end TypeScript com Node.js e Fastify em um único aplicativo, organizado em domínio, aplicação, infraestrutura e interfaces HTTP. A linguagem é livre no enunciado; esta escolha favorece contratos tipados e reutilização de conhecimento.

Supabase: PostgreSQL como banco relacional e Supabase Auth como emissor de JWT. SQL e migrações versionadas; driver PostgreSQL no servidor. Relações, constraints e transações justificam o banco para OS, orçamento e estoque. Ferramentas propostas: Vitest para testes, cobertura V8 e OpenAPI/Swagger para contratos.

Deploy pretendido: adaptar o aplicativo HTTP único ao runtime Node da Vercel, sem distribuir regras em Edge Functions. Esse mapeamento e as versões exatas devem ser confirmados na documentação oficial antes de implementar. Fixar versões e lockfile; não há compatibilidade de deploy validada neste pacote.

Fluxo técnico: cliente HTTP -> interface REST -> casos de uso -> domínio -> repositórios PostgreSQL. Auth emite token; API valida assinatura, issuer, audience e validade e aplica autorização. O navegador do portal não acessa dados operacionais nem contém credenciais.

A aplicação é monolítica por organização e unidade de código/deploy; a execução gerenciada pode ter múltiplas instâncias. Estado e locks ficam no banco, nunca na memória de uma instância. Conexões ao banco devem usar pool compatível com o runtime, limite de concorrência e transações curtas.

Alternativas: microserviços descartados por extrapolar o enunciado e aumentar operação. Acesso direto do navegador ao banco descartado para impedir contorno das regras. Hospedagem persistente em container é alternativa se a adaptação à Vercel falhar; deve ser avaliada antes de alterar o alvo solicitado.

Execução local: API em container e PostgreSQL em Compose; autenticação por uma instância Supabase local documentada ou projeto dedicado de desenvolvimento. O README final deve permitir iniciar todo o ambiente, incluindo autenticação, sem depender de credenciais pessoais. Compose e Dockerfile do back-end ainda serão implementados.

03 / Domínio e propriedade dos dados

Contextos propostos: Cadastros (cliente e veículo), Atendimento (OS e orçamento), Catálogo/Estoque (serviços, peças e movimentações) e Identidade/Acesso (usuários e perfis). São módulos do mesmo monólito, não serviços independentes.

Agregado OrdemServico: controla status, itens, versão concorrente e histórico. Orçamento: versão, preços congelados, total e decisão. Estoque: saldo físico, reservas e movimentos por peça. Entidades: Cliente, Veiculo, ServicoCatalogo, Peca, OrdemServico, Orcamento e MovimentoEstoque.

Objetos de valor: DocumentoFiscal, Placa, Dinheiro em BRL e Quantidade. IDs UUID; datas em UTC; apresentação local explícita. Quantidades de peças inteiras e insumos fracionários devem ter unidades declaradas e escala decimal definida.

Event Storming exigido: criação da OS, acompanhamento da OS e gestão de peças/insumos. Em cada mapa registrar ator, comando, política, agregado e evento. Eventos propostos: OrdemRecebida, DiagnosticoIniciado, OrcamentoDisponibilizado, OrcamentoAprovado, OrcamentoRejeitado, ExecucaoIniciada, ServicoFinalizado e VeiculoEntregue.

Glossário: OS é o atendimento de um veículo; orçamento é uma proposta versionada; aprovação autoriza os itens daquela versão; reserva separa disponibilidade sem baixar saldo físico; consumo registra a baixa física; entrega encerra o atendimento.

04 / Fluxo de negócio e máquina de estados

01 Recebida: atendente identifica cliente, vincula veículo, registra solicitações e abre OS. Comando iniciar diagnóstico leva a Em diagnóstico.

02 Em diagnóstico: técnico detalha serviços, peças e quantidades. Publicar orçamento calcula total, congela preços e leva a Aguardando aprovação.

03 Aguardando aprovação: cliente consulta somente sua OS e aceita ou rejeita a versão atual. Aprovar reserva peças em transação; a OS permanece aguardando início com decisão APROVADA. Iniciar execução exige aprovação válida e reserva suficiente e leva a Em execução.

Rejeição: decisão REJEITADA mantém o status Aguardando aprovação e bloqueia execução. Reabrir diagnóstico invalida a proposta anterior e retorna a Em diagnóstico, gerando nova versão para aprovação. Não inventar um sétimo status como requisito FIAP.

04 Em execução: registrar início e consumo das peças reservadas. Finalizar exige serviços concluídos e reservas resolvidas; leva a Finalizada. Não permitir alteração silenciosa do escopo aprovado.

05 Finalizada: registrar conclusão e calcular duração. A entrega física confirmada pelo atendente leva a Entregue. 06 Entregue: histórico preservado e alterações operacionais bloqueadas.

Reparo adicional: proposta de MVP é impedir alteração de orçamento aprovado e exigir revisão controlada antes de novo trabalho. Retorno durante execução e tratamento de serviços parcialmente executados precisam ser definidos e testados antes de incluir essa extensão.

05 / Regras de negócio verificáveis

RN01 Cliente: normalizar e validar CPF/CNPJ por algoritmo vigente e formato; documento único. Não usar dados reais em seed ou vídeo. Mudanças de formatos oficiais devem ser verificadas antes da implementação.

RN02 Veículo: validar formatos de placa aplicáveis e normalizar; exigir marca, modelo e ano plausível. Placa única no cadastro ativo; transferência de titularidade preserva o cliente original de cada OS.

RN03 OS: exigir cliente, veículo e ao menos uma solicitação. Número público único e ID interno distintos. Catálogos inativos não entram em novas OS; registros históricos não são apagados por exclusão de cadastro.

RN04 Valores: armazenar valores monetários em centavos inteiros ou decimal exato; nunca ponto flutuante. Total = soma de quantidade x preço dos serviços e materiais. Preços e descrições ficam congelados na versão do orçamento; desconto fora do MVP.

RN05 Aprovação: decisão vinculada a versão, cliente e instante. Aprovação de versão antiga retorna conflito. Repetir a mesma aprovação não gera segunda reserva. Rejeição registra decisão; execução permanece bloqueada.

RN06 Estoque: disponível = físico - reservado. Aprovação reserva todos os materiais ou falha integralmente. Locks por peça em ordem determinística e constraints impedem saldo negativo. Duas OS disputando a última peça não podem ambas aprovar.

RN07 Consumo: transformar reserva em baixa física uma única vez, com referência ao item da OS. Estorno é novo movimento auditável; não editar ou apagar movimentos anteriores. Retirar peça do orçamento libera reserva quando aplicável.

RN08 Transições: aceitar somente as transições documentadas. Estado, versão e pré-condições são revalidados dentro da transação. Toda mudança registra autor, origem, destino e data.

RN09 Concorrência: atualização da OS exige versão esperada; conflito retorna 409. Criação, aprovação e consumo aceitam chave de idempotência vinculada ao usuário e hash do pedido; reutilização com conteúdo diferente retorna conflito.

RN10 Tempo médio: considerar OS com execução finalizada; duração = finalized_at - execution_started_at. Exibir quantidade da amostra e período. OS ainda abertas não contam como zero. MVP usa tempo corrido e documenta que não desconta pausas.

RN11 Acesso: administrador gere catálogo e usuários; atendente gere cadastros, orçamento e entrega; técnico diagnostica e executa; cliente consulta e decide somente sobre as próprias OS. Perfis são controlados no servidor, nunca em metadados editáveis pelo usuário.

RN12 Auditoria: guardar histórico sem CPF, token ou senha em logs. Uma OS entregue não pode ser excluída. CRUD permite exclusão física apenas quando não há referências; caso contrário usar inativação com erro explicativo.

RN13 Falhas: rollback integral em aprovação/reserva e consumo. Nenhuma chamada externa ocorre segurando locks. Disponibilizar orçamento pela API atende ao fluxo; envio de e-mail real é extensão, com caixa de saída e retentativa se adotado.

06 / Dados, contratos e segurança

Tabelas propostas: customers (document único), vehicles (plate única, customer_id), service_catalog (preço e ativo), parts (unidade, preço, físico, reservado), service_orders (cliente, veículo, status, versão e datas), order_service_items, order_part_items, quotes (order_id, versão e total), quote_decisions, stock_movements, status_history, user_profiles e idempotency_keys.

Constraints: FKs explícitas; quantidades positivas nos itens; preços não negativos; físico >= reservado >= 0; unicidade de versão por OS e de consumo por referência de item. Índices por cliente/data, status/data e referências de estoque. Transações de orçamento e estoque pertencem ao mesmo banco.

API /api/v1: CRUD /customers, /vehicles, /services e /parts; GET/POST /orders; GET /orders/{id}; POST /orders/{id}/diagnosis, /quotes, /start, /finish e /deliver; POST /quotes/{id}/decision; GET /me/orders e /me/orders/{id}; GET /metrics/execution-time; POST /parts/{id}/movements.

Contratos: payloads e respostas tipados, paginação e filtros limitados; erros 400 para entrada inválida, 401 sem autenticação, 403 sem perfil, 404 para recurso ausente ou alheio, 409 para concorrência/regra conflitante e 500 sanitizado. Cada erro inclui código estável e request_id, sem detalhes internos.

Autorização por recurso é obrigatória mesmo com JWT válido. Cliente não pesquisa CPF para acessar outra OS. Vínculo auth_user_id -> customer_id é criado por fluxo confiável e não por autoatribuição do navegador.

Banco operacional em schema privado não exposto na Data API, com role de aplicação limitada. Para qualquer tabela exposta: grants explícitos, RLS e policies específicas, testadas com dois usuários distintos. Chaves privilegiadas nunca no HTML ou no Git.

Segredos somente em variáveis protegidas de cada ambiente. .env.example contém nomes e valores fictícios. Logs com request_id, erro, duração e operação; monitorar falhas de aprovação, conflitos e esgotamento de conexões. Não registrar payloads pessoais.

Publicação documental contém somente dados sintéticos. Conteúdo público inclui arquitetura e evidências sanitizadas; não inclui dump de clientes, credenciais, URLs assinadas ou tokens. Portal estático não oferece formulário de upload público.

07 / Etapas de construção e critérios de saída

D+0 - Requisitos: checklist completo do enunciado e dúvidas anotadas. Saída: todo requisito tem responsável, artefato e critério verificável.

D+1 - Domínio: Event Storming dos três fluxos, glossário, agregados e transições. Saída: rejeição, estoque e autorização têm comportamento definido.

D+2 - Base: versões confirmadas, lockfile, API inicial, migrações, autenticação, Docker e Compose. Saída: novo clone inicia o ambiente conforme README e executa um teste de integração.

D+3 - Cadastros: CRUD, validações e perfis. Saída: happy path e entradas inválidas passam; usuário sem perfil é bloqueado.

D+4 - OS e orçamento: criação, itens, cálculo e histórico. Saída: preços congelados, total correto e proposta versionada.

D+5 - Aprovação e estoque: reserva atômica, rejeição, idempotência e concorrência. Saída: teste simultâneo da última peça prova apenas uma reserva bem-sucedida.

D+6 - Execução e entrega: transições, consumo, consulta do cliente e tempo médio. Saída: jornada completa demonstrável e acesso a OS alheia bloqueado.

D+7 - Qualidade: cobertura >=80% nos domínios críticos, integração real, análise de vulnerabilidades e correção dos achados relevantes. Saída: relatórios associados ao commit.

D+8 - Documentação e ensaio: exportação DDD, README validado por instalação limpa, OpenAPI e vídeo de até 15 minutos. Saída: roteiro ensaiado com dados sintéticos.

D+9 - Fechamento: conferir identificação, URLs públicas, branch/hash, PDF final e submissão. Planejamento relativo proposto, não promessa de execução: ajustar D+0 para concluir antes de 03/11/2026 às 23h59.

08 / Plano de testes e evidências

T01 Documentos/placas: aceitar formatos válidos e rejeitar inválidos. T02 Orçamento: quantidades, arredondamento exato, versão e preços congelados. T03 Estados: cobrir todas as transições permitidas e recusadas.

T04 Aprovação: versão obsoleta, decisão duplicada e rejeição. T05 Estoque: saldo insuficiente, rollback, estorno e duas transações concorrentes para a última peça. Usar PostgreSQL real nos testes de integração, não mocks para provar atomicidade.

T06 JWT: ausente, expirado, assinatura incorreta, issuer/audience inválidos e perfil inadequado. T07 Isolamento: cliente A tenta consultar/aprovar OS de B. T08 Idempotência: repetição retorna o mesmo efeito e payload divergente gera 409.

T09 Indicador: período sem amostra, OS aberta, finalizada e duração conhecida. T10 Fluxo completo: cadastro até entrega, incluindo leitura pela API do cliente. T11 Instalação limpa: Docker Compose, migrações, seed e Swagger acessíveis.

Cobertura: medir domínio e casos de uso críticos, registrar escopo e exclusões; configurar falha da verificação abaixo de 80%. Não inflar número excluindo regras importantes. Relatório deve incluir commit e comando executado.

Vulnerabilidades: dependências, segredos e análise estática; registrar ferramenta/versão/data/commit, resultados e tratamento ou justificativa dos achados. Este pacote não contém resultado executado.

Matriz de rastreabilidade: cada requisito -> regra -> endpoint/caso de uso -> teste -> evidência -> commit. Guardar evidências em docs/evidencias sem dados pessoais. No portal, publicar somente arquivos realmente produzidos.

09 / Documentação DDD e roteiro do vídeo

DDD: publicar quadro Miro ou equivalente acessível sem aprovação manual e exportar Event Storming e diagramas para docs/ddd. Incluir atores, comandos, eventos, políticas, agregados, entidades, objetos de valor, contextos, linguagem ubíqua e ligação com classes/casos de uso.

Vídeo proposto: 0:00-1:00 identificação e proposta; 1:00-3:00 arquitetura, banco e camadas; 3:00-5:00 Event Storming e DDD; 5:00-10:00 jornada real de OS, materiais, orçamento, aprovação e status; 10:00-12:00 JWT, isolamento e erros; 12:00-14:00 testes, cobertura e vulnerabilidades; 14:00-15:00 execução, Swagger e commit.

O vídeo precisa ser público no YouTube e corresponder à versão final. O portal incorpora somente o ID real informado em assets/config.js; enquanto vazio, mostra pendência e não carrega um vídeo fictício. Manter também link direto no PDF oficial.

PDF final: primeira página com curso, fase, grupo, nomes completos, RMs, Discord e tecnologias; demais páginas com todos os repositórios, branch/hash, vídeo, DDD e evidências. Validar links em janela sem sessão e preservar disponibilidade até o fim da revisão de notas.

10 / Estrutura, operação e conclusão

portal/index.html e portal/assets: portal estático pronto para consulta local. portal/pdfs: plano e fluxo de negócio gerados neste pacote. docs/planejamento: fonte editorial e checklist; docs/ddd e docs/evidencias: destinos das futuras evidências. backend/src/domain, application, infrastructure e interfaces: estrutura reservada, sem implementação.

backend/tests/unit e integration: testes futuros. backend/db/migrations e seeds: futuros scripts versionados. entrega/: futuro PDF oficial. README.md explica abertura, configuração e integração ao Git existente. Não criar um .git aninhado.

Git: copiar o subdiretório para o repositório existente, ler AGENTS.md e políticas locais, verificar status e remotes, criar alterações por PR e preservar arquivos anteriores. Nenhum commit, push, repositório remoto ou deploy foi realizado neste pacote.

Publicação Vercel do portal: escolher TECH CHALENGE/portal como diretório raiz e saída estática. Confirmar previamente o tratamento do caminho com espaço no painel. O back-end terá projeto de deploy próprio apontando ao mesmo repositório, depois de implementado e validado.

Checklist final: todos os links reais; nenhum segredo; nenhuma alegação de teste sem relatório; RM/Discord preenchidos; GitHub e YouTube públicos; DDD externo acessível; exportações no Git; commit consistente; PDF oficial submetido na FIAP. O portal não envia a atividade automaticamente.

Fontes: enunciado Fase 1 - Tech Challenge lido no portal FIAP; atividade https://on.fiap.com.br/mod/assign/view.php?id=645805&returnto=conteudocurso . Referências a validar na implementação: https://vercel.com/docs/functions ; https://supabase.com/docs/guides/auth ; https://supabase.com/docs/guides/database . Decisões de stack e regras adicionais neste documento são propostas de solução.